

Robin: Source-Code Architecture

A code-level companion to `robin-guide.md`. Everything below is traced through the released repo (github.com/Future-House/robin). The `robin/` Python package is ~3.5k lines of straight-line async orchestration — there is no agentic loop in Robin itself; the only LLM-driven agents (Crow/Falcon/Finch) live behind the hosted Edison API. This confirms the paper's claim that the stabilized workflow was "collapsed into a deterministic notebook."

Repo Layout

<code>robin/</code>		the orchestrator package (importable)
<code>__init__.py</code>	12 lines	public API: <code>experimental_assay</code> , <code>therapeutic_candidates</code> , <code>data_analysis</code> , <code>RobinConfiguration</code>
<code>configuration.py</code>	330 lines	<code>RobinConfiguration</code> , <code>Prompts</code> , <code>AgentConfig</code> ; Edison + LiteLLM clients
<code>prompts.py</code>	835 lines	every system/user prompt string (data-only, no logic)
<code>assays.py</code>	277 lines	PHASE 1 – <code>experimental_assay()</code> : pick the assay/strategy
<code>candidates.py</code>	548 lines	PHASE 2 – <code>therapeutic_candidates()</code> : propose + rank drugs
<code>analyses.py</code>	202 lines	<code>data_analysis()</code> : Finch flow-cytometry/RNA-seq + insight distillation
<code>multitrajectory_runner.py</code>	293 lines	<code>Step/StepConfig/MultiTrajectoryRunner</code> – parallel Finch driver
<code>utils.py</code>	975 lines	Edison calls, BTL ranking, file I/O, report formatting
<code>robin_demo.ipynb</code>		minimal run (defaults 3/3/5, no data analysis)
<code>robin_full.ipynb</code>		paper run (5/10/30 + data-analysis refinement loop)
<code>examples/</code>		11 disease notebooks + pre-generated <code>example_outputs/</code>
<code>robin_output/</code>		two sample dAMD runs (the actual paper outputs)
<code>pyproject.toml</code>		deps: <code>edison-client</code> ≥ 0.11, <code>fhlmi</code> , <code>fhaviary</code> , <code>choix</code> , <code>anthropic</code> , <code>openai</code>
<code>.env.example</code>		<code>EDISON_API_KEY</code> , <code>OPENAI_API_KEY</code>

Dependency aliases worth knowing: `lmi` = `fhlmi` (FutureHouse LiteLLM wrapper), `aviary.core` = `fhaviary`, `edison_client` = the hosted-platform SDK exposing `JobNames`, `EdisonClient`, `TaskRequest/TaskResponse`.

High-Level Data Flow



Every box labeled o4-mini is `configuration.llm_client.call_single(...)`. Every Edison box is an HTTP round-trip via `call_platform (utils.py:70)` or `MultiTrajectoryRunner.run_pipeline (multitrajectory_runner.py:199)`. The orchestrator is the glue Python in between.

Module / Function Map

`configuration.py`

- `get_default_llm_config (62)` — hard-codes o4-mini LiteLLM entry, `api_key` pulled from `OPENAI_API_KEY` at instantiation (timeout 300s).
- `Prompts (86)` — Pydantic model, one field per prompt string defaulting to the constant in `prompts.py`. `validate_all_prompts (145)` statically checks each template's {placeholders} against an expected set via the `_get_prompt_args` regex (71) — a fail-fast guard so a malformed prompt edit raises at construction, not mid-run.

- AgentConfig (242) — which Edison job each step uses. Defaults: assay lit-search = CROW, assay report = CROW, candidate lit-search = CROW, candidate report = FALCON (244-259). Falcon is used only for the deep candidate reports.
- RobinConfiguration (266) — central config object. Defaults num_queries=3, num_assays=3, num_candidates=5 (272-282) — the *demo* config, NOT the paper's 5/10/30. run_folder_name auto-set to {disease[:70]}_{timestamp} (301). Lazy edison_client (309, reads EDISON_API_KEY env first) and llm_client (321, LiteLLMModel) properties. get_da_client (327) returns a MultiTrajectoryRunner.

assays.py — experimental_assay(configuration) -> goal_string (27). Five steps:

1. o4-mini generates num_queries broad pathology queries, split on <> (54-59).
2. call_platform runs them through CROW; save_crow_files dumps each review (72-84).
3. o4-mini proposes num_assays {strategy_name, reasoning} JSON objects; parsed with a JSON-array fallback regex (111-129).
4. create_assay_hypothesis_queries (154) builds one report prompt per assay; CROW writes a structured eval per assay (184-188).
5. Ranking: uniformly_random_pairs(num_assays) -> run_comparisons (o4-mini judge) -> choix.ilsr_pairwise(num_assays, games, alpha=0.1) (219-232); sort by strength_score, take iloc[0] as top_experimental_assay (243). synthesize_candidate_goal (249) turns that assay + disease into the candidate_generation_goal string returned to the caller.

candidates.py — therapeutic_candidates(goal, configuration, experimental_insights=None) (30). Mirrors Phase 1 but for drugs, and is the refinement entry point:

- Query gen uses 2*num_queries ("double_queries", split half-disease/half-mechanism) (71-78).
- If experimental_insights is passed, two prompt appendages are spliced in: EXPERIMENTAL_INSIGHTS_APPENDAGE onto the query-gen system message (55-69) and EXPERIMENTAL_INSIGHTS_FOR_CANDIDATE_GENERATION onto the candidate-gen user message (148-162). Output folders/CSVs get an _experimental suffix (109-120, 317-328, 531-543).
- Candidate parsing is brittle text scraping, not JSON: split on <CANDIDATE END>, require <CANDIDATE START>, regex ^([A-Z_]+): to slot CANDIDATE/HYPOTHESIS/REASONING (174-235).
- FALCON writes the deep report per candidate (307); format_final_report (313) makes a *second* o4-mini pass per report to reformat citations into APA 7th (utils.py:821).
- Ranking: extract_candidate_info_from_folder re-reads the saved .txt reports (regex Proposal for... Overview for the name, utils.py:894), then the same run_comparisons -> games-validation gauntlet (403-460) -> choix.ilsr_pairwise(n_items, games, alpha=0.1) (469) -> ranked_therapeutic_candidates[_experimental].csv.

analyses.py — data_analysis(data_path, data_analysis_type, goal, configuration) -> dict (17). The Finch driver:

- Picks analysis_query/consensus_query from prompts by data_analysis_type ("flow_cytometry" | "RNA_seq") (26-28).
- Step 1 analysis_step (55): job job-futurehouse-data-analysis-crow-high, R, max_steps=30, timeout=15min, parallel=PARALLEL_ANALYSIS=5 -> 5 parallel Finch trajectories, each writes flow_results.csv.
- Step 2 consensus_step (67): a single trajectory that meta-analyzes the 5 result files -> consensus_results.csv.

- `read_and_process_csv` → HTML table (capped 30k chars) → o4-mini "data interpretation" call whose output is split on `<>` into exactly 4 fields (130-170): `drugs_in_data`, `analysis_summary`, `questions_raised`, `mechanistic_insights`.
- A second o4-mini "followup" call proposes the next assay (181-188). `analysis_summary` is suffixed with the literal "...DO NOT SUGGEST THESE DRUGS AGAIN." (190-195) so the next candidate round won't re-propose tested drugs. Returns the `experimental_insights` dict.

`multitrajectory_runner.py` — the generic parallel-trajectory engine:

- `StepConfig` (20): `language` (default PYTHON), `max_steps`, `timeout`, `eval`. `Step` (33): job name, prompt template, `input_files/output_files` maps, parallel count, 8-char `step_id`, optional `prompt_generator`. `cot_prompting/format_prompt` (76-101) optionally wrap the query with the CoT + language guidelines.
- `MultiTrajectoryRunner` (104): `add_step`, `run_pipeline` (199). Per step: upload input files (210), build `RuntimeConfig` with `environment_config={eval, language}` (217), `_create_task_requests` (158), then `client.arun_tasks_until_done(...)` (233) — the blocking call that runs N trajectories on Edison. Computes a `success_rate`, downloads each output file with an `_idx` suffix per trajectory (254-274), runs optional `post_process`, dumps `results_*.json`.
- Key detail (187-195): when `parallel>1` *without* a `prompt_generator`, it duplicates the *identical* `TaskRequest` N times. So Finch's 5 trajectories run the *same prompt* — diversity comes purely from LLM sampling stochasticity, exactly the "harvest the variance" design.

`utils.py` — the workhorse:

- `call_platform` (70) — submit one Edison task per query (`create_task`, 85), poll all concurrently (`gather_results/poll_for_task_completion`, 31-67, 5s interval, 6000s overall timeout). On success it digs the PaperQA2 citations out of `verbose_task_result.environment_frame["state"]["state"]["response"]["answer"]["references"]` (181-183) and returns `{results, count, has_errors}`.
- `uniformly_random_pairs` (422) — the sampling policy. `n_games = min(300, C(n,2))` (440) with a **fixed seed=621** (426) → deterministic pair selection. For $n \leq 25$, $C(n,2) \leq 300$ so it's a full round-robin; for $n > 25$ it samples 300 unordered pairs.
- `process_comparison_pair` (584) — one judge call: builds a "Candidate 1 vs Candidate 2" user prompt, `client.call_single`, then brace-matches and `json.loads` the response, requiring keys `{Analysis, Reasoning, Winner, Loser}` (642-650).
- `run_comparisons` (698) — fans all pairs out under an `asyncio.Semaphore(100)`, `tqdm_asyncio.gather`, writes the per-pair Winner/Loser/Analysis/Reasoning CSV.
- `processing_ranking_output` (456) — parses the judge's Winner/Loser (`name, id`) tuple strings (regex + `ast.literal_eval` fallback) into (`winner_id, loser_id`) "Game Score" tuples — the input to `choix`.
- Misc: `save_crow_files/save_falcon_files` (240/290), `format_assay_ideas/format_candidate_ideas` (use `<|>` field separators, 353/374), `extract_candidate_info_from_folder` (868), `format_final_report` APA pass (858), `read_and_process_csv` (946).

Control Flow of One Full Cycle (robin_full.ipynb)

```

config = RobinConfiguration(disease_name="dry age-related macular degeneration",
                           num_queries=5, num_assays=10, num_candidates=30)

goal = await experimental_assay(configuration=config)          # PHASE 1
└ 5 queries → CROW → 10 assays → CROW reports
  → 10 hypotheses, C(10,2)=45 pairs (≤300 ⇒ full round-robin)
  → o4-mini judge → choix BTL → top assay
  → synthesize goal string

await therapeutic_candidates(candidate_generation_goal=goal,    # PHASE 2 (round 1)
                             configuration=config)
└ 10 queries → CROW → 30 candidates → FALCON deep reports (+APA)
  → 30 hypotheses, C(30,2)=435 > 300 ⇒ 300 sampled pairs
  → o4-mini judge → choix BTL → ranked_therapeutic_candidates.csv
                                | humans test top ~5 at bench
                                ▼
data_path = "AG4/"; data_analysis_type = "flow_cytometry"
experimental_insights = await data_analysis(data_path, data_analysis_type,  # PHASE 3
                                             goal=goal, configuration=config)
└ 5 Finch trajectories (R, gating+MFI) → consensus trajectory
  → consensus_results.csv → o4-mini interpret(4 fields) + followup
  → {analysis_summary(+ "don't re-suggest"), mechanistic_insights,
     questions_raised, followup_suggestions}

await therapeutic_candidates(candidate_generation_goal=goal,    # PHASE 2 (round 2)
                             configuration=config,
                             experimental_insights=experimental_insights)
└ same pipeline, prompts now conditioned on prior wet-lab results;
  outputs land in *_experimental files

```

The whole notebook uses top-level await (the cells are async); there is no `asyncio.run`. `goal` is computed once and reused across both candidate rounds. The "loop" is a single manual hop wired by the human recalling `therapeutic_candidates` with the insights dict — there is no automatic iteration construct in the code.

Paper Concept → Code Location

Paper concept	Code location
Two-phase hypothesis generation	assays.py:27 (assay) ; candidates.py:30 (drug)
Literature-ground before judging	CROW/FALCON reports become the judged text: assays.py:184 ; candidates.py:307
LLM-judge pairwise comparison (Winner/Loser JSON)	utils.py:process_comparison_pair:584 prompt: ASSAY_/CANDIDATE_RANKING_PROMPT_FORMAT
BTL ranking from pairwise wins	choix.ilsr_pairwise(..., alpha=0.1) assays.py:232 ; candidates.py:469
Incomplete-block sampling (≤25 full, >25 → 300 pairs)	utils.py:uniformly_random_pairs:422 n_games = min(300, C(n,2)), seed=621
Meta-generated judge rubric	baked into prompts.py CANDIDATE_RANKING_SYSTEM_PROMPT (evidence>MoA>safety>feasibility>novelty, :691)
Finch parallel-consensus (N trajectories → consensus)	analyses.py:17 + multitrajectory_runner.py PARALLEL_ANALYSIS=5 (Step 1) + 1 consensus (Step 2)
Result-conditioned refinement (in-context, not fine-tuning)	candidates.py:55-69 & 148-162 (insight appendages) driven by experimental_insights dict from analyses.py
Deterministic notebook (no agent loop in the orchestrator)	the whole robin/ package is straight-line async; agency is confined to Edison-hosted Crow/Falcon/Finch
"don't re-test known drugs"	analyses.py:190 literal prompt suffix
Crow=concise, Falcon=deep	JobNames.CROW vs FALCON in AgentConfig (config.py:244)
Crow→job-futurehouse-paperqa2	(confirmed in demo-notebook logs)
Falcon→job-futurehouse-paperqa2-deep	

Code-vs-Paper Surprises and Gaps

1. **The judge is the same o4-mini, not Claude 3.7 Sonnet.** The paper's headline judge-validation numbers (88.4% self-consistency, 7.25/10 expert overlap) used Claude 3.7 Sonnet. But the released code has a *single* `llm_client`, and `run_comparisons` is called with `client=configuration.llm_client` (`assays.py:223`, `candidates.py:375`) — i.e. o4-mini does both generation *and* judging. `grep` finds no `claude/sonnet/anthropic/gemini` anywhere in `robin/`. (`anthropic` is a dependency, so one could set `llm_name` to a Claude model, but that would swap generation too — there is no separate judge-model knob.) The meta-generated judge *prompts* themselves are present (`CANDIDATE_RANKING_SYSTEM_PROMPT`), but not run against the paper's judge model out of the box.
2. **Default config (3/3/5) ≠ paper config (5/10/30).** `RobinConfiguration` defaults are a cheap demo (`config.py:272-282`); the paper scale is set explicitly only in `robin_full.ipynb`. `robin_demo.ipynb` even runs 3/3/5 to completion (its logs show "Starting generation of 5 therapeutic candidates").
3. **Finch is 5 trajectories, not 10.** `PARALLEL_ANALYSIS = 5` (`analyses.py:13`). The paper describes "up to 10" and reports an 8-trajectory RNA-seq consensus; the released code hard-codes 5 for the first step plus 1 consensus pass. The N parallel trajectories run the *identical* prompt (`multitrajectory_runner.py:187-195`) — consensus diversity is pure sampling noise, by design.
4. **data_analysis is hard-wired to the flow-cytometry case.** Although `data_analysis_type` selects the *query text* for "flow_cytometry" vs "RNA_seq" (`prompts.py:279-362`), the Step plumbing in `analyses.py` is flow-specific: input is mapped to a hard-coded "flow_250508/" folder (with a "change this to your input folder" comment, :59) and Step 1's `output_files` expects `flow_results.csv` (:60) — but the RNA-seq analysis query instructs the agent to emit `dea_results.csv`. So an out-of-the-box RNA-seq run would mis-match the download path; reproducing the paper's RNA-seq result requires editing this function.
5. **The "ranking" is mostly defensive plumbing.** `therapeutic_candidates` Step 5 spends ~150 lines (`candidates.py:387-548`) validating and salvaging `choix` inputs — out-of-range IDs, malformed game tuples, length mismatches, `choix` exceptions each get their own logged fallback CSV. The judge's free-text (`name`, `id`) output is parsed by `regex/ast` (`utils.py:456`), so the brittleness of LLM-as-structured-output is handled with parsing guards rather than a constrained-decoding API.
6. **Two o4-mini passes per candidate report.** Beyond generation and judging, every Falcon report is run through a *separate* APA-citation reformatting call (`format_final_report`, `utils.py:858`) before it is saved and later judged — an easily-missed extra LLM cost per candidate.
7. **"Edison" vs "FutureHouse" naming drift.** The client package is `edison_client` and links point to `platform.edisonscientific.com`, but the underlying job strings are still `job-futurehouse-paperqa2 / -deep / -data-analysis-crow-high`, and the demo logs say "FutureHouse platform". README notes Crow/Falcon are "now called 'Literature'". The Crow/Falcon/Finch names in the paper survive only as the `JobNames` enum aliases.